



Ministère de l'Enseignement Supérieur et de la Recherche
Scientifique
Université de Carthage
Institut supérieur des technologies de l'information et de la
communication

Chapitre 6 : Les chaînes de caractères

Plan

1. Introduction
2. Déclaration
3. Accès aux éléments d'une chaîne
4. Précédence alphabétique et lexicographique
5. Manipulation des chaînes de caractères
6. Exercices

1. Introduction

Il n'existe pas de type spécial chaîne ou string en C. une chaîne de caractères est traitée comme un tableau de caractères.

2. Déclaration

La déclaration d'une chaîne de caractères se fait de la manière suivante :

```
char NomVariable [Longueur];
```

Lors de la déclaration, nous devons indiquer l'espace à réserver en mémoire pour le stockage de la chaîne. La représentation interne d'une chaîne de caractères est terminée par le symbole '\0'. Ainsi, pour un texte de n caractères, nous devons prévoir n+1 octets.

Exemple

```
char TXT [10]; /*déclaration sans initialisation*/  
char TXT[10] = "BONJOUR !"; /*déclaration avec initialisation*/  
char TXT[] = {'B','O','N','J','O','U','R',' ','!'}; /*déclaration avec initialisation*/
```

0	1	2	3	4	5	6	7	8	9
'B'	'O'	'N'	'J'	'O'	'U'	'R'	' '	'!'	'\0'

Remarque :

- Les chaînes de caractères constantes sont indiquées entre guillemets. La chaîne de caractères vide est alors: ""
- Dans les chaînes de caractères, on peut utiliser toutes les séquences d'échappement définies comme caractères constants: "Ce \n texte \n sera écrit sur 3 lignes."
- Le symbole " peut être représenté à l'intérieur d'une chaîne par la séquence d'échappement \" : "Affichage de \"guillemets\" \n"

3. Accès aux éléments d'une chaîne

- L'accès à un élément d'une chaîne de caractères peut se faire de la même façon que l'accès à un élément d'un tableau. En déclarant une chaîne par:

```
char A[5];
```

on a défini un tableau A avec cinq éléments, auxquels on peut accéder par: A[0], A[1], ... , A[5]

Exemple

A[0]	A[1]	A[2]	A[3]	A[4]	A[5]
'H'	'E'	'L'	'L'	'O'	'\0'

4. Précédence alphabétique et lexicographique

Précédence alphabétique des caractères : La précédence des caractères dans l'alphabet d'une machine est dépendante du code de caractères utilisé. Pour le code ASCII, on peut constater l'ordre suivant : ... ,0,1,2, ... ,9, ... ,A,B,C, ... ,Z, ... ,a,b,c, ... ,z, ...

Relation de précédence : A partir de la précédence alphabétique des caractères, on peut déduire une relation de précédence 'est inférieur à' sur l'ensemble des caractères. Ainsi, on peut dire que '0' est inférieur à 'Z' et noter '0' < 'Z' car dans l'alphabet de la machine, le code du caractère '0' (ASCII: 48) est inférieur au code du caractère 'Z' (ASCII: 90).

Précédence lexicographique des chaînes de caractères : En se basant sur cette relation de précédence alphabétique des caractères, on peut définir une précédence lexicographique pour les chaînes de caractères.

Remarque : La chaîne vide "" précède lexicographiquement toutes les autres chaînes.

Exemples

"ABC" précède "BCD" car 'A' < 'B'

"ABC" précède "B" car 'A' < 'B'

"Abc" précède "abc" car 'A' < 'a'

"ab" précède "abcd" car "" précède "cd"

" ab" précède "ab" car ' ' < 'a'

(le code ASCII de ' ' est 32, et le code ASCII de 'a' est 97)

5. Manipulation des chaînes de caractères

Les bibliothèques de fonctions de C contiennent une série de fonctions spéciales pour le traitement de chaînes de caractères.

A. Les fonctions de <stdio.h>

Comme nous l'avons déjà vu, la bibliothèque <stdio.h> nous offre des fonctions qui effectuent l'entrée et la sortie des données (printf, scanf, puts, gets).

Affichage de chaînes de caractères

`printf` avec le spécificateur de format `%s` permet d'intégrer une chaîne de caractères dans une phrase.

Exemple

```
char TXT [20] ;
printf("%s", TXT);
//ou
puts (TXT);
```

Lecture de chaînes de caractères

`scanf` avec le spécificateur `%s` permet de lire un mot isolé à l'intérieur d'une suite de données du même ou d'un autre type.

Exemple

```
char LIEU[25];
int JOUR, MOIS, ANNEE;
printf("Entrez lieu et date de naissance : \n");
scanf("%s %d %d %d", LIEU, &JOUR, &MOIS, &ANNEE);
```

On remarque dans cet exemple que malgré que la fonction `scanf` a besoin des adresses de ses arguments, et à la différence des variables entières `JOUR`, `MOIS` et `ANNEE`, la variable `LIEU`, qui est une chaîne de caractères, n'a pas été précédée par le symbole '&'. Ceci est dû au fait que le nom d'une chaîne de caractères est le représentant de l'adresse de son premier caractère. Notons que pour la lecture d'une chaîne de caractères contenant des espaces, il est conseillé d'utiliser la fonction `gets` car `scanf` arrêtera la lecture de la variable avec le premier caractère d'espacement rencontré.

Exemple

```
char Nom[20];
printf("Donnez votre nom :");
scanf("%s ", Nom);
printf("Vous vous appelez : %s ", Nom);
char Nom[20];
printf("Donnez votre nom :");
gets(Nom) ;
printf("Vous vous appelez : %s ", Nom);
```

Si l'utilisateur saisit « Jean Paul » :

- dans le cas du scanf, le programme affichera : Vous vous appelez : Jean
- dans le cas du gets, le programme affichera : Vous vous appelez : Jean Paul

B. Les fonctions de <string.h>

int strlen(s)	fournit la longueur de la chaîne sans compter le '\0' final <pre>int x, y ; char ch[]="imprimante" ; x = strlen ("Disque") ; y = strlen(ch) ;</pre> Ce programme renvoie la valeur 6 dans x et la valeur 10 dans y.
void strcpy(s, t)	copie t vers s (y compris le caractère \0). <pre>char ch[]="imprimante", chaine[20], str[5]; strcpy(chaine, ch) ; /* affecter "imprimante" à la variable chaine */ strcpy(str, ch) ; /*erreur de débordement : "imprimante" nécessite 11 octets (10+1) alors que le système n'a alloué pour str que 5 octets. */</pre>
void strcat(s, t)	ajoute t à la fin de s <pre>char ch[30]="Bonjour", chaine[]=" tout le monde"; strcat(ch, chaine) ;</pre> ch contenant initialement "Bonjour", contiendra après la concaténation la chaîne "Bonjour tout le monde".
int strcmp(s, t)	compare <s> et <t> lexicographiquement et fournit un résultat: négatif si <s> précède <t> zéro si <s> est égal à <t> positif si <s> suit <t>
void strncpy(s, t, n)	copie au plus n caractères de t vers s <pre>strncpy (ch, "Bonjour", 3 ; ch contiendra la chaîne</pre>

	"Bon". strncpy (ch, "Bonjour", 10) ch contiendra la chaîne "Bonjour".
void strncat(s, t, n)	ajoute au plus <n> caractères de <t> à la fin de <s> char ch[10]= "Bon" ; strncat (ch, "Bonjour", 3) ch contiendra la chaîne "BonBon".
void strrev(s)	Inverser la chaîne s

C. Les fonctions de <stdlib.h>

La bibliothèque <stdlib> contient des déclarations de fonctions pour la conversion de nombres en chaînes de caractères et vice-versa.

Conversion de Chaîne en Nombre

int atoi(<s>)	retourne la valeur numérique représentée par <s> comme int
long atol(<s>)	retourne la valeur numérique représentée par <s> comme long
double atof(<s>)	retourne la valeur numérique représentée par <s> comme float/double

Règles générales pour la conversion:

Les espaces au début d'une chaîne sont ignorés

La conversion s'arrête au premier caractère non convertible

Pour une chaîne non convertible, les fonctions retournent zéro

Conversion de nombre en une chaîne

itoa (<int>, <s>,)

ltoa (<long>, <s>,)

ultoa (<unsigned long>, <s>,)

Chacune de ces trois procédures convertit son premier argument en une chaîne de caractères qui sera ensuite attribuée à <s>. La conversion se fait dans la base .

Exemple

```
char ch[10], str [10];
```

```
itoa(5, ch, 2);
```

```
itoa(189, str, 12);
```

ch aura la chaîne "101" (5=101 en base 2).

str aura la chaîne "139" (189=139 en base 12).

D. Les fonctions de <ctype.h>

Les fonctions de <ctype.h> servent à classifier et à convertir des caractères.

Une fonction de <ctype.h> prend en paramètre un caractère.

Les fonctions de classification suivantes fournissent un résultat du type int différent de zéro, si la condition respective est vérifiée, sinon zéro.

<u>La fonction:</u>	<u>retourne une valeur différente de zéro,</u>
int isupper(<c>)	si <c> est une majuscule ('A'...'Z')
int islower(<c>)	si <c> est une minuscule ('a'...'z')
int isdigit(<c>)	si <c> est un chiffre décimal ('0'...'9')
int isalpha(<c>)	si islower(<c>) ou isupper(<c>)
int isalnum(<c>)	si isalpha(<c>) ou isdigit(<c>)
int isxdigit(<c>)	si <c> est un chiffre hexadécimal ('0'...'9' ou 'A'...'F' ou 'a'...'f')
int isspace(<c>)	si <c> est un signe d'espacement (' ', '\t', '\n', '\r', '\f')

Les fonctions de conversion suivantes fournissent une valeur du type char

char tolower(<c>)	retourne <c> converti en minuscule si <c> est une majuscule
char toupper(<c>)	retourne <c> converti en majuscule si <c> est une minuscule

6. Tableaux de chaînes de caractères

Souvent, il est nécessaire de mémoriser une suite de mots ou de phrases dans des variables. Il est alors pratique de créer un tableau de chaînes de caractères, ce qui allégera les déclarations des variables et simplifiera l'accès aux différents mots (ou phrases).

A. Déclaration, initialisation et mémorisation

Un tableau de chaînes de caractères correspond à un tableau à deux dimensions du type char, où chaque ligne contient une chaîne de caractères.

Lors de la déclaration il est possible d'initialiser toutes les composantes du tableau par des chaînes de caractères constantes:

```
char JOUR[7][10]= {"lundi", "mardi", "mercredi", "jeudi", "vendredi",
"samedi", "dimanche"};
```

JOUR:

'l'	'u'	'n'	'd'	'i'	'\0'		
'm'	'a'	'r'	'd'	'i'	'\0'		
'm'	'e'	'r'	'c'	'r'	'e'	'd'	'i' '\0'
...	...	...	...	...	...	...	...
'd'	'i'	'm'	'a'	'n'	'c'	'h'	'e' '\0'

B. Accès aux différentes composantes

Accès aux chaînes

Il est possible d'accéder aux différentes chaînes de caractères d'un tableau, en indiquant simplement la ligne correspondante.

Exemple

L'exécution des instructions suivantes:

```
char JOUR[7][9]= {"lundi", "mardi", "mercredi", "jeudi",
"vendredi", "samedi", "dimanche"};
```

```
int I = 2;
```

```
printf("Aujourd'hui, c'est %s \n", JOUR[I]);
```

```
printf("Hier, c'était %s \n", JOUR[I-1]);
```

affichera les phrases:

Aujourd'hui, c'est mercredi

Hier, c'était mardi

Affectation

Des expressions comme JOUR[I] représentent l'adresse du premier élément d'une chaîne de caractères. Donc, on ne doit jamais essayer de modifier une telle adresse par une affectation directe.

~~JOUR[4] = "Friday";~~

L'attribution d'une chaîne de caractères à une composante d'un tableau de chaînes se fait en général à l'aide de la fonction strcpy:

Exemple

La commande :

```
strcpy(JOUR[4], "Friday");
```

changera le contenu de la 5e composante du tableau JOUR de "vendredi" en "Friday".

Accès aux caractères

Evidemment, il existe toujours la possibilité d'accéder directement aux différents caractères qui composent les chaînes du tableau.

Exemple

```
for (I=0; I<7; I++)
```

```
printf("%c ", JOUR[I][0]);
```

va afficher les premières lettres des jours de la semaine:

l m m j v s d

Exercice 1

Ecrire un programme qui permet d'introduire le nom et le prénom de l'utilisateur et affiche :

- La longueur totale du nom et du prénom.
- Compare le nom et le prénom lexicographiquement.
- Copie la moitié du nom et du prénom dans une troisième variable et l'affiche.
- Inverse le nom et l'affiche.
- Affiche le nom en caractères majuscules et le prénom en caractères minuscules.

```
#include<stdio.h>
#include<string.h>
void main()
{
    char nom[20], prenom[20], ch[20];
    char a;
    printf("donner nom ");
    gets(nom);
    printf("donner prenom ");
    gets(prenom);
    /*Compare le nom et le prénom lexicographiquement*/
    if (strcmp(nom, prenom)==0)
        printf("nom %s et prenom %s sont égaux lexicographiquement\n", nom, prenom);
    else
        if (strcmp(nom, prenom)<0)
            printf("nom %s précède lexicographiquement prenom %s\n",
nom, prenom);
        else
            printf("nom %s suit lexicographiquement prenom %s\n ",
nom, prenom);
    /*Copie la moitié du nom et du prénom dans une troisième
variable et l'affiche.*/
    strncpy(ch, nom, strlen(nom)/2 );
    strncat(ch, prenom, strlen(prenom)/2 );
    puts(ch);
}
```

```

    /*Inverse le nom et l'affiche.*/
    puts(strrev(nom));

    /*Affiche le nom en caractères majuscules et le prénom en
    caractères minuscules*/
    for(int i=0; nom[i]!='\0';i++)
    {
        if((nom[i] >= 'a') && (nom[i] <= 'z')) {
            nom[i] = nom[i] -32;
        }
    }
    puts(nom);

    /*Affiche le prenom en caractères majuscules et le prénom
    en caractères minuscules*/
    for(int i=0; prenom[i]!='\0';i++)
    {
        if((prenom[i] >= 'A') && (prenom[i] <= 'Z')) {
            prenom[i] = prenom[i] +32;
        }

    }

    puts(prenom);
}

```

Exercice 2

Soit Tnom un tableau de caractères à 2 dimensions déclaré comme suit : char Tnom [50][30]

Le tableau Tnom est destiné à contenir des noms de personnes, exemple :

'm'	'a'	'n'	'e'	'l'	'\0'		...		
'a'	'l'	'i'	'\0'						
'r'	'o'	'c'	'h'	'd'	'i'	'\0'			
...		..							
'a'	'h'	'm'	'e'	'd'	'\0'				

Travail à faire :

A- Pour chacun des traitements suivants, écrire le code en C la fonction qui permet de le réaliser

1- Saisir n, le nombre de noms à stocker dans le tableau

2- Remplir le tableau Tnom par n noms sachant qu'un nom doit contenir au minimum 3 caractères

3- Afficher le plus long nom contenu dans le tableau

4- Saisir un nom au clavier et afficher sa première occurrence dans Tnom s'il existe.

5- Saisir un nom au clavier et le supprimer du tableau Tnom

6- Mettre toutes les premières lettres de chaque nom en majuscule et les autres caractères en minuscule

7- Afficher tous les noms

B- Ecrire le programme principal qui permet de faire appel aux différentes fonctions appelées précédemment.

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include<stdbool.h>
void controle (int *n)
{do{
    printf ("donner le nombre de ligne :");
    scanf("%d", n);
}while ((*n<=0) || (*n>50));
}
void saisi (char mat[][30], int n)
{int m;
int j;
int i;
for (i=0; i<n;i++) {
do{
printf("saisir un nom ") ;
scanf ("%s",mat[ i]) ;
}while ((strlen(mat[i])<3) || (strlen(mat[i])>30));
/*j=0;
while(j<strlen(mat[i]))
{
do
{printf ("%c", mat[i][j]);
```

```

        mat[i][j]=getchar();

        }while          ((isalpha(mat[i][j]==0))          ||
(isspace(mat[i][j]==0)));
        j++;
    }*/
}
}

void affiche(char mat[][30], int *n)
{
    for (int i=0; i<*n; i++)
        puts(mat[i]);
    printf("\n");
}

void long_nom(char mat[][30], int n)
{int indice_max;
    int max =strlen (mat[0]);
    for (int i=1; i<n; i++ )
        if (strlen (mat[i])>max)
        {
            indice_max=i;
        }

    printf ("%s \n", mat[indice_max]);
}

int exsite (char mat[][30], int *n, char nom [30])
{int p=-1,j=0;
bool trouve=false;
int i=0;
while ((i<*n)&& (trouve==false))
{
    if ((strcmp(mat[i],nom) == 0))
    {
        trouve=true;
    }
}
}

```

```

        p=i;
    }
    i++;
}
return p;
}

void delete(char mat[][30], int *n, char nom[30])
{int p=exsite (mat, n, nom);
    if(p!=-1)
    {
        for (int i=p; i<*n-1; i++)
        {
            strcpy(mat[i],mat[i+1]);

        }

        *n=*n-1;

    }

}

void majescule (char mat[][30], int *n)
{
    for (int i=0; i<*n; i++)
    {
        mat[i][0]=toupper(mat[i][0]);
    }
}

int main()
{
    char mat[50][30];
    int n, m;
    char nom [30], ns[30];

```

```

    controle(&n);
// printf("%d",n);
    saisi(mat, n);
    printf("le plus long nom \n");
    long_nom(mat, n);
    printf ("liste des noms\n");
    affiche(mat, &n);
    printf ("donner le nom à chercher :");
    scanf ("%s", nom);
    int p= exsite(mat, &n, nom);
    printf ("nom existe dans la position %d \n",p);
    printf ("donner le nom à supprimer :");
    scanf("%s", ns);
    delete(mat, &n,ns );
    printf ("liste des noms après suppression :\n");
    affiche(mat, &n);
    majescule(mat, &n);
    affiche(mat, &n);

}

```